

Class 8: Classes and Object-Oriented Python

Data-driven Systems Engineering

Agenda

- What object-oriented programming actually buys you.
- Defining a class: `__init__`, `self`, attributes, and methods.
- Class attributes versus instance attributes, and private attributes by convention.
- Classes you already use every day: `RandomForestClassifier`, `StandardScaler`, `Flask`.
- Inheritance and why scikit-learn is built on it.

Learning goals

- Explain what a class, an instance, an attribute, and a method are.
- Define a small class with a constructor and at least one method.
- Distinguish class attributes from instance attributes.
- Recognize that objects you already call — models, scalars, apps — are instances of classes.
- Describe what inheritance provides, using scikit-learn's estimator design as the example.

Course Map and Running Cases



Today in the course

Focus: Object-oriented programming: bundling data and behavior together.

Bridge: Every function so far has stood alone; a class lets a group of functions share state on purpose.

Fraud case

Every time the fraud pipeline writes `RandomForestClassifier(...)` or `StandardScaler()`, it is creating an object — the fraud model itself is an instance of a class.

Pasteurization case

The pasteurization service instantiates `Flask(__name__)` once and calls methods on that one object for the rest of its life.

Course thread: You have been using classes since Class 5 without naming them; today gives that experience its vocabulary.

What object-oriented programming is

- OOP centers on objects that hold both data and the functions that act on that data, in one place.
- A **class** is the blueprint; an **instance** is one concrete object built from that blueprint.
- A class has **attributes** (the data an object carries) and **methods** (the functions it can call on itself).
- Almost everything in Python is an object — strings, lists, and functions all have attributes and methods, whether or not you defined their class.

Defining a minimal class

```
class SensorReading:
    def __init__(self, name, value):
        self.name = name
        self.value = value

    def describe(self):
        return f"{self.name} = {self.value}"

r = SensorReading("T", 72.0)
print(r.describe())
```

- class starts the definition; `__init__` is the constructor, run automatically when an instance is created.
- `self` refers to the specific instance being built or used; every method takes it as its first parameter.

Classes you are already instantiating

```
scaler = StandardScaler()  
cls = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1)  
app = Flask(__name__)
```

- All three lines, from 'streamlit/modeling.py' and 'Class6_model_api.py', create an **instance** of a class you did not write yourself.
- `n_estimators`, `random_state`, and `__name__` are arguments passed straight into that class's `__init__`.

Calling methods on an object

```
cls.fit(X_train_scaled, y_train)
y_pred = cls.predict(X_test_scaled)
app.run(host="0.0.0.0", port=5001, debug=True)
```

- `.fit(...)`, `.predict(...)`, and `.run(...)` are methods: functions that belong to the object and act on its internal state.
- After `.fit()`, `cls` holds a trained model *inside itself* — that is exactly what attributes are for.

Class attributes versus instance attributes

- An **instance attribute** belongs to one object; it is usually set inside `__init__` with `self.name = ....`
- A **class attribute** is defined directly in the class body and is shared by every instance unless overridden.
- In the example above, every `SensorReading` has its own name and value; a class attribute would instead be the same for all of them, such as a shared unit label.
- `RandomForestClassifier`'s constructor arguments (`n_estimators`, `random_state`) become instance attributes on `cls` the moment it is created.

Private attributes, by convention

- Python has no strict access control; “private” is a naming convention, not enforced by the interpreter.
- A single leading underscore, `_value`, signals “internal use, please do not touch from outside.”
- Preferring a method over direct attribute access (a getter, in spirit) keeps the internal representation free to change later.
- scikit-learn follows this convention itself: attributes set only after `.fit()`, like `feature_names_in_`, end with an underscore to mark them as fitted state.

Classes as modules, modules of classes

- A class can live in its own .py file, then be imported exactly like the functions in Class 7.
- Larger projects group related classes into a package: a directory of modules with clear responsibilities.
- This repository keeps things flatter — one class per external library call — but the same organizing principle applies as a project's custom classes grow.

Inheritance

```
class Sensor:
    def __init__(self, name):
        self.name = name

    def read(self):
        raise NotImplementedError

class TemperatureSensor(Sensor):
    def read(self):
        return f"{self.name}: reading in degrees C"
```

- Inheritance lets one class reuse and extend another's attributes and methods.
- `TemperatureSensor(Sensor)` inherits everything from `Sensor` and only overrides what needs to differ.
- `super().__init__(...)` calls the parent's constructor when the child needs to extend it, not replace it.

Inheritance you rely on without seeing it

- Every scikit-learn estimator, `RandomForestClassifier` included, inherits from common base classes so that `fit`, `predict`, and parameter handling behave the same way across the whole library.
- This shared base is exactly what lets you swap one model class for another and keep calling `.fit()` and `.predict()` unchanged — a theme Class 11 returns to directly.
- Buitinck et al. (2013) document this design choice as central to scikit-learn's API.

How the repo already uses today's building blocks

Repo activity

- 'streamlit/modeling.py' instantiates `StandardScaler` and `RandomForestClassifier`, then calls methods on each.
- 'Class6_model_api.py' instantiates `Flask` once at module load and reuses that one object for every route.
- Find one more class instantiation anywhere in 'streamlit/' or 'pasteurization/' and name its constructor arguments.

Question

'RandomForestClassifier' and 'StandardScaler' are both instantiated with no custom class ever written for them. When does it actually pay off to write your own class, rather than just calling functions?

- Pushes past “because OOP is best practice” toward a concrete trade-off: shared state and repeated behavior versus added structure.

Papers and materials

Buitinck et al. (2013), API design for machine learning software; scikit-learn documentation; Flask documentation.

From This Class to the Next

What stays with us

- Classes bundle data and behavior; instances are the concrete objects built from that blueprint.
- `self`, `__init__`, and methods are the syntax; attributes carrying state are the payoff.
- You were already using classes — scikit-learn's and Flask's — well before today's vocabulary caught up.

Project use

Before writing more script-level code for your project, ask whether related state and behavior should be grouped into a class instead.

Next connection

Class 9 turns back to data itself: loading, indexing, and light cleaning with pandas on a real dataset.

Course thread: Objects hold state; next class is about the state — the data — those objects will eventually operate on.